

Universidad de las fuerzas armadas Espe
Estudiante: Christian Padilla.
Asignatura: Fundamentos de la programación
Nrc: 28561
Fecha: lunes 25 de mayo de 2026
Ingeniera: Jenny Ruiz

Ejercicio 1: Vector al revés

Enunciado: Desarrolle un programa que lea las N componentes de un vector y las escriba a continuación en orden inverso al introducido. El valor de N se introduce por teclado y se supone que será igual o menor a 10.

1. Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción
Vector de 10 componentes	vec	Real (float o double)	Variable
Número de componentes	N	Entero (int)	Variable
Contador	i	Entero (int)	Variable.
Uno	1	Entero (int)	Constante
Cero	0	Entero (int)	Constante

2. Algoritmo

```
#include <stdio.h>
int main() {
    float vec[10];
    int N;
    int i;
    do {
        printf("Ingrese la cantidad de elementos (1 a 10): ");
        scanf("%d", &N);
    } while (N < 1 || N > 10);
    printf("\n--- Entrada de Datos ---\n");
```

```

for (i = 0; i < N; i++) {
    printf("Ingrese el elemento %d: ", i + 1);
    scanf("%f", &vec[i]);
}
printf("\nEl vector en orden inverso es:\n");
for (i = N - 1; i >= 0; i--) {
    printf("%.2f ", vec[i]);
}
printf("\n");
return 0;
}

```

3. Prueba de escritorio

Paso / Operación	Variable N	Variable i	Condición del ciclo	vec[i]	Salida
1. Leer N	3	-	-	-	-
2. Ciclo Lectura (i=0)	3	0	0 < 3 (Verdadero)	vec[0] = 4.5	Ingrese elemento 1:
3. Ciclo Lectura (i=1)	3	1	1 < 3 (Verdadero)	vec[1] = 8.1	Ingrese elemento 2:
4. Ciclo Lectura (i=2)	3	2	2 < 3 (Verdadero)	vec[2] = 3.0	Ingrese
5. Fin ciclo lectura	3	3	3 < 3 (Falso)	-	Termina lectura.
6. Ciclo Inverso (i=2)	3	2	2 > \ 0 (Verdadero)	Lee vec[2]	3
7. Ciclo Inverso (i=1)	3	1	1 > \ 0 (Verdadero)	Lee vec[1]	8.1

8. Ciclo Inverso (i=0)	3	0	$0 \geq 0$ (Verdadero)	Lee <code>vec[0]</code>	4.5
9. Fin ciclo inverso	3	-1	$-1 \geq 0$ (Falso)	-	Fin del programa.

Ejercicio 2. Norma de un vector al cuadrado

Enunciado: Desarrolle un programa que lea las n componentes reales de un vector `vec` y calcule la norma al cuadrado: $m = \text{vec}^T \times \text{vec} = \text{suma de } \text{vec}(i)^2$. El tamaño n se introduce por teclado y puede suponerse igual o menor que 10

1. Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción
Vector de 10 componentes	<code>vec</code>	Real (<code>float</code>)	Variable: Almacena los números reales ingresados.
Número de componentes del vector	<code>n</code>	Entero (<code>int</code>)	Variable: Almacena el tamaño real del vector ($n \geq 10$).
Contador	<code>i</code>	Entero (<code>int</code>)	Variable: Índice del ciclo para controlar el recorrido del vector.
Norma al cuadrado	<code>res</code>	Real (<code>float</code>)	Variable: Acumulador que guarda la suma de los cuadrados.

Uno	1	Entero (int)	Constante: Límite inferior para la validación del tamaño.
-----	---	-----------------	---

2. Algoritmo

```
#include <stdio.h>
int main() {
    float vec[10];
    int n;
    int i;
    float res;
    do {
        printf("Ingrese el tamaño del vector (1 a 10): ");
        scanf("%d", &n);
    } while (n < 1 || n > 10);
    res = 0.0;
    printf("\n--- Entrada de Componentes ---\n");
    for (i = 0; i < n; i++) {
        printf("vec[%d]: ", i + 1);
        scanf("%f", &vec[i]);
        res = res + (vec[i] * vec[i]);
    }
    printf("\n--- Resultado ---\n");
    printf("La norma al cuadrado del vector es: %.4f\n", res);
    return 0;
}
```

3. Prueba de escritorio

Paso / Operación	Variabl e n	Variabl e i	vec[i] (Dato)	Operación matemátic a	Acumulado r res
---------------------	----------------	----------------	----------------------	-----------------------------	--------------------

1. Inicialización	3	-	-	-	0.0
2. Ciclo lectura e i=0	3	0	1.5	0.0 + (1.5)^2 = 0.0 + 2.25	2.25
3. Ciclo lectura e i=1	3	1	2.0	2.25 + (2.0)^2 = 2.25 + 4.0	6.25
4. Ciclo lectura e i=2	3	2	3.0	6.25 + (3.0)^2 = 6.25 + 9.0	15.25
5. Fin del ciclo	3	3	-	Condición 3 < 3 es Falsa	15.25

Ejercicio 3. Vector con término general dado

Enunciado: Sea la sucesión $v_k = k^2 + 3$. Desarrolle un programa que lea el número n de componentes que se quieren de la sucesión y las almacene en un vector vec, de forma que $vec(i) = v_i$. Puede asumir que n será menor o igual a 100.

1. Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción

Vector de 100 componentes	<code>vec</code>	Real (<code>float</code>)	Variable: Almacena los valores calculados de la sucesión.
Número de componentes del vector	<code>n</code>	Entero (<code>int</code>)	Variable: Cantidad de términos solicitados por el usuario (\$n \leq 100\$).
Contador	<code>i</code>	Entero (<code>int</code>)	Variable: Controla el ciclo de cálculo y recorrido posicional.
Variable para el valor matemático k	<code>k</code>	Entero (<code>int</code>)	Variable auxiliar: Representa el índice matemático del término (inicia en 1).
Uno	<code>1</code>	Entero (<code>int</code>)	Constante: Límite mínimo para la validación y base de conteo matemático.

2. Algoritmo

```
#include <stdio.h>
int main() {
    float vec[100];
    int n;
    int i;
    int k;
    do {
        printf("Ingrese el numero de componentes (1 a 100): ");
        scanf("%d", &n);
```

```

} while (n < 1 || n > 100);
for (i = 0; i < n; i++) {
    k = i + 1;    vec[i] = (k * k) + 3; // v_k = k^2 + 3
}
printf("\n--- Sucesion Generada (vec) ---\n");
for (i = 0; i < n; i++) {
    printf("vec[%d] = %.2f\n", i, vec[i]);
}
return 0;
}

```

3. Prueba de escritorio

Paso / Operación	Variab le n	Variab le i	Variab le k (i+1)	Fórmula matemática (k ² +3)	Estado del Vector (vec)
1. Leer n	3	-	-	-	-
2. Ciclo i=0	3	0	1	1 ² + 3 = 1 + 3 = 4	vec[0] = 4.0
3. Ciclo i=1	3	1	2	2 ² + 3 = 4 + 3 = 7	vec[1] = 7.0
4. Ciclo i=2	3	2	3	3 ² + 3 = 9 + 3 = 12	vec[2] = 12.0

5. Fin ciclo	3	3	-	Condición 3 < 3 es Falsa	Fin del procesamiento.
--------------	---	---	---	-----------------------------	------------------------

Ejercicio 4. Comprobar si dos valores pertenecen a un vector

Enunciado. Realice un algoritmo que lea dos números enteros por teclado y determine si ambos valores forman parte de un vector de enteros previamente definido de dimensión 15.

1. Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción
Número de componentes del vector	N	Entero (int)	Constante: Dimensión fija del vector (15).
Vector de 15 números	vec	Entero (int)	Variable: El arreglo con los datos previamente definidos.
Primer dato de entrada	num1	Entero (int)	Variable: Primer número a buscar, ingresado por el usuario.
Segundo dato de entrada	num2	Entero (int)	Variable: Segundo número a buscar, ingresado por el usuario.

Variable auxiliar (Bandera 1)	aux1	Entero (int)	Variable: Bandera para registrar si se encontró num1 (0 = No, 1 = Sí).
Variable auxiliar (Bandera 2)	aux2	Entero (int)	Variable: Bandera para registrar si se encontró num2 (0 = No, 1 = Sí).
Contador	i	Entero (int)	Variable: Índice para recorrer las 15 posiciones del vector.
Cero	0	Entero (int)	Constante: Valor de inicialización para las banderas (falso).
Uno	1	Entero (int)	Constante: Valor de activación para las banderas (verdadero).

2. Algoritmo

```
#include <stdio.h>
```

```
int main() {
```

```
    int vec[15] = {5, 12, 8, 23, 42, 1, 7, 19, 30, 15, 3, 88, 14, 9, 50};
```

```
    int num1, num2;
```

```
    int aux1 = 0;
```

```
    int aux2 = 0;
```

```
    int i;
```

```
    printf("Ingrese el primer numero entero a buscar: ");
```

```
    scanf("%d", &num1);
```

```
    printf("Ingrese el segundo numero entero a buscar: ");
```

```

scanf("%d", &num2);
for (i = 0; i < 15; i++) {
    if (vec[i] == num1) {
        aux1 = 1;
    }
    if (vec[i] == num2) {
        aux2 = 1;
    }
}
printf("\n--- Resultado de la Búsqueda ---\n");
if (aux1 == 1 && aux2 == 1) {
    printf("EXITO: Ambos valores (%d y %d) forman parte del vector.\n",
num1, num2);
} else {
    printf("RECHAZADO: Uno o ambos valores NO se encuentran en el
vector.\n");
    // Detalle informativo para el usuario
    if (aux1 == 0) printf("- El numero %d no fue encontrado.\n", num1);
    if (aux2 == 0) printf("- El numero %d no fue encontrado.\n", num2);
}

return 0;
}

```

3. Prueba de escritorio

Paso / Operación	Variable i	Elemento vec[i]	Condición vec[i]==num1	Condición vec[i]==num2	Estado aux1	Estado aux2
<i>Inicialización</i>	-	-	-	-	0	0

Ciclo pos <i>i</i> = 0	0	5	5 == 7 (Falso)	5 == 99 (Falso)	0	0
... (pos 1 a 5)	-	-	(Falsos)	(Falsos)	0	0
Ciclo pos <i>i</i> = 6	6	7	7 == 7 (Verdadero)	7 == 99 (Falso)	1	0
-	-	-	-	(Falsos)	1	0
Evaluación Final	Fin ciclo	-	-	Condición: (aux1==1 Y aux2==1) (1==1 Y 0==1) FALSO		

Ejercicio 5. Vector de factoriales

Enunciado. Dado un vector Vec que contiene los primeros 15 números naturales, calcula un vector fact con sus factoriales y muéstrelo por pantalla. Considere el problema de almacenamiento a partir de factoriales grandes.

1.Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción

Vector de 15 números naturales	Vec	Entero (int)	Variable: Contiene los números del 1 al 15.
Vector de 15 factoriales	fact	Entero largo (long long)	Variable: Almacena los factoriales calculados. <i>Se usa un tipo grande por seguridad.</i>
Contador del vector principal	i	Entero (int)	Variable: Controla el índice del elemento actual en Vec y fact .
Contador para el cálculo matemático	j	Entero (int)	Variable: Multiplica sucesivamente para hallar el factorial.
Cero	0	Entero (int)	Constante: Límite inferior de los índices en C.
Uno	1	Entero (int)	Constante: Valor inicial para la multiplicación acumulada.
Quince	15	Entero (int)	Constante: Dimensión fija de los arreglos.

2. Algoritmo

```
#include <stdio.h>
int main() {
```

```

int Vec[15] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};
long long fact[15];
int i, j;
long long f_acumulado;
for (i = 0; i < 15; i++) {
    f_acumulado = 1;
    for (j = 1; j <= Vec[i]; j++) {
        f_acumulado = f_acumulado * j;
    }
    fact[i] = f_acumulado;
}
printf("%-10s %-20s\n", "Numero (N)", "Factorial (N!)");
printf("-----\n");
for (i = 0; i < 15; i++) {
    printf("%-10d %-20lld\n", Vec[i], fact[i]);
}
return 0;
}

```

3. Prueba de escritorio

Iteración Externa (i)	Valor Vec[i]	Iteración Interna (j)	Operación (f_acumulado * j)	Variable f_acumulado	Destino fact[i]
<i>Inicio</i>	-	-	-	1	-
i = 0	1	j = 1	1 * 1	1	fact[0] = 1

i = 1	2	j = 1	1 \times 1	1	fact[1] = 2
		j = 2	1 \times 2	2	
i = 2	3	j = 1	1 \times 1	1	fact[2] = 6
		j = 2	1 \times 2	2	
		j = 3	2 \times 3	6	

Ejercicio 6. Suma de componentes de una matriz

Enunciado. Halle la suma de las componentes de una matriz cuya dimensión y componentes se leen por teclado y por filas. Suponga que el número de filas N y columnas M son menores o iguales a 10.

1. Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción
Matriz de 10x10 componentes	mat	Real (float)	Variable: Almacena los datos bidimensionales introducidos.

Número de filas de la matriz	N	Entero (int)	Variable: Dimensión de filas elegida por el usuario ($N \geq 10$).
Número de columnas de la matriz	M	Entero (int)	Variable: Dimensión de columnas elegida por el usuario ($M \geq 10$).
Contador externo (Filas)	i	Entero (int)	Variable: Controla el índice de la fila actual.
Contador interno (Columnas)	j	Entero (int)	Variable: Controla el índice de la columna actual.
Suma de las componentes	res	Real (float)	Variable: Acumulador donde se almacena la sumatoria.
Uno	1	Entero (int)	Constante: Límite mínimo para las validaciones de entrada.

2. Algoritmo

```
#include <stdio.h>
int main() {
    float mat[10][10];
    int N, M;
    int i, j;
    float res;
    do {
        printf("Ingrese el numero de filas (1 a 10): ");
        scanf("%d", &N);
```

```

} while (N < 1 || N > 10);
do {
    printf("Ingrese el numero de columnas (1 a 10): ");
    scanf("%d", &M);
} while (M < 1 || M > 10);
res = 0.0;
printf("\n--- Entrada de Datos por Filas ---\n");
for (i = 0; i < N; i++) {
    for (j = 0; j < M; j++) {
        printf("Matriz[%d][%d]: ", i + 1, j + 1);
        scanf("%f", &mat[i][j]);
        res = res + mat[i][j];
    }
}
printf("\n--- Resultado ---\n");
printf("La suma de todas las componentes de la matriz es: %.2f\n", res);
return 0;
}

```

3. Prueba de escritorio

Fila (i)	Columna (j)	Condición de Ciclos	mat[i][j] (Lectura)	Operación (res + mat[i][j])	Acumulado r res
<i>Inicio</i>	-	-	-	-	0.0
0	0	0 < 2 \text{ e } 0 < 2 (V)	mat[0][0] = 5.0	0.0 + 5.0	5.0

0	1	$0 < 2 \setminus \text{e} \}$ 1 < 2 (V)	<code>mat[0][1] = 3.0</code>	5.0 + 3.0	8.0
0	2	2 < 2 (Falso)	<i>Termina fila 0</i>	-	8.0
1	0	$1 < 2 \setminus \text{e} \}$ 0 < 2 (V)	<code>mat[1][0] = 1.5</code>	8.0 + 1.5	9.5
1	1	$1 < 2 \setminus \text{e} \}$ 1 < 2 (V)	<code>mat[1][1] = 2.0</code>	9.5 + 2.0	11.5
1	2	2 < 2 (Falso)	<i>Termina fila 1</i>	-	11.5
2	-	2 < 2 (Falso)	<i>Termina Matriz</i>	-	11.5

Ejercicio 6. Matriz traspuesta

Enunciado. Dada una matriz A de dimensión N x M, cuyas dimensiones reales y componentes se leen por teclado y por filas, calcule e imprima la matriz B de dimensión M x N, tal que $B = A^T$. Suponga que filas y columnas de A son menores o iguales a 10.

1. Tabla de objetos

Objeto	Nombre	Tipo de dato	Rol / Descripción
Matriz de datos	mat	Real (float)	Variable: Almacena los números de la matriz (10 $\times$ 10 máx).
Vector de máximos	Vmax	Real (float)	Variable: Almacena el valor máximo hallado en cada fila (dimensión 10 máx).
Cantidad de filas	N	Entero (int)	Variable: Número de filas que usará el usuario ($N \leq 10$).
Cantidad de columnas	M	Entero (int)	Variable: Número de columnas que usará el usuario ($M \leq 10$).
Contador externo	i	Entero (int)	Variable: Controla el índice de las filas de la matriz y del vector.
Contador interno	j	Entero (int)	Variable: Controla el índice de las columnas de la matriz.
Máximo provisional	max_actual	Real (float)	Variable auxiliar: Guarda temporalmente el mayor valor de la fila en evaluación.

2. Algoritmo

```
#include <stdio.h>
```

```
int main() {
    float mat[10][10];
    float Vmax[10];
    int N, M;
    int i, j;
    float max_actual;

    do {
        printf("Ingrese el numero de filas (1 a 10): ");
        scanf("%d", &N);
    } while (N < 1 || N > 10);

    do {
        printf("Ingrese el numero de columnas (1 a 10): ");
        scanf("%d", &M);
    } while (M < 1 || M > 10);

    printf("\n--- Entrada de Datos ---\n");
    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++) {
            printf("Matriz[%d][%d]: ", i + 1, j + 1);
            scanf("%f", &mat[i][j]);
        }
    }

    for (i = 0; i < N; i++) {
        max_actual = mat[i][0];

        for (j = 1; j < M; j++) {
            if (mat[i][j] > max_actual) {
                max_actual = mat[i][j]; // Actualización del máximo de la fila
            }
        }
    }
}
```

```

    Vmax[i] = max_actual;
}

printf("\n--- Resultados (Vector Vmax) ---\n");
for (i = 0; i < N; i++) {
    printf("El valor maximo de la Fila %d es: %.2f\n", i + 1, Vmax[i]);
}

return 0;
}

```

3. Prueba de escritorio

Fila (i)	Columna (j)	Valor mat[i][j]	Inicialización / Comparación	Variable max_actual	Vector Vmax[i]
0	-	-	Asume primer elemento <code>mat[0][0]</code>	1.5	-
0	1	4.8	4.8 > 1.5 Verdadero	4.8	-
0	2	2.2	2.2 > 4.8 Falso	4.8	-
<i>Fin j</i>	-	-	Guarda en el vector	4.8	<code>Vmax[0] = 4.8</code>

1	-	-	Asume primer elemento <code>mat[1][0]</code>	9.1	-
1	1	3.0	$3.0 > 9.1$ Falso	9.1	-
1	2	12.4	$12.4 > 9.1$ Verdadero	12.4	-
<i>Fin j</i>	-	-	Guarda en el vector	12.4	<code>Vmax[1] = 12.4</code>